



VIT[®]
BHOPAL
www.vitbhopal.ac.in

PROJECT REPORT

NEWS CLASSIFIER SYSTEM

NAME – AYUSHI DUBEY

REG NO. - 25MIM10174

BRANCH – INTEGRATED MTECH AIML

COURSE – FUNDAMENTALS IN AI&ML(CSA2001)

INTRODUCTION

In today's digital world, a massive amount of news content is generated every second. Categorizing this news manually is time-consuming and inefficient. This project focuses on building an automated system that classifies news headlines into predefined categories using Machine Learning techniques.

The system uses Natural Language Processing (NLP) to process textual data and a classification model to predict the appropriate category. Additionally, a graphical user interface (GUI) is implemented using Tkinter to make the system user-friendly.

MOTIVATION OF THE PROJECT

The motivation behind this project is to simplify the process of organizing and analyzing news data. With the rapid growth of digital content, automated classification helps in:

Reducing manual effort

Improving accuracy

Enhancing user experience

Enabling faster decision-making

PROBLEM STATEMENT

Manual classification of news articles is inefficient and prone to errors. There is a need for an automated system that can accurately categorize news headlines into relevant domains such as sports, politics, business, etc.

OBJECTIVE OF THE PROJECT

- To build a machine learning model for classifying news headlines
 - To implement text preprocessing techniques
 - To design a user-friendly GUI
 - To allow both single input and bulk dataset classification
 - To generate output in a structured format
-

EXISTING METHODS

Existing methods include:

1. Manual categorization
 2. Rule-based systems
 3. Keyword-based classification
-

PROS AND CONS OF THE STATED METHODS

Pros:

- Easy to implement
- No need for training data

Cons:

- Low accuracy
 - Not scalable
 - Cannot handle complex language patterns
 - Time-consuming
-

HARDWARE AND SOFTWARE REQUIREMENTS

Hardware:

- Minimum 4GB RAM
- Basic computer/laptop

Software:

- Python
- Pandas
- Scikit-learn
- NLTK
- Tkinter

METHODOLOGY AND GOAL

Methodology:

1. Data Collection
2. Data Preprocessing
3. Feature Extraction (TF-IDF)
4. Model Training (Logistic Regression)
5. Prediction
6. GUI Integration

Goal:

To create an efficient and interactive system for automated news classification.

FUNCTIONAL MODULES DESIGN AND ANALYSIS

- Data Preprocessing Module → Cleans and prepares text
- Model Training Module → Trains ML model
- Prediction Module → Predicts category
- GUI Module → User interaction
- File Handling Module → Handles CSV input/output

ALGORITHM DEVELOPMENT

Step-by-step Algorithm:

1. Convert text to lowercase
 2. Remove special characters
 3. Remove stopwords
 4. Apply TF-IDF vectorization
 5. Train Logistic Regression model
 6. Predict category for new input
-

SOFTWARE ARCHITECTURAL DIAGRAM

User Input → Text Cleaning → TF-IDF → ML Model → Prediction → Output

↑
Dataset

CODING

The system is implemented in Python using libraries such as Pandas, NLTK, and Scikit-learn. The GUI is developed using Tkinter for user interaction.

OUTPUT

Single headline classification shown in GUI

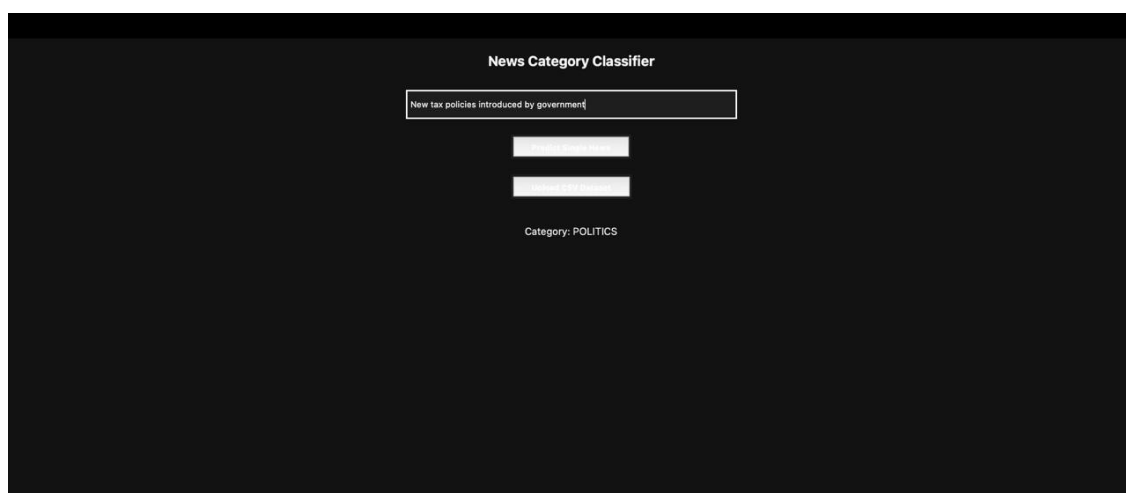
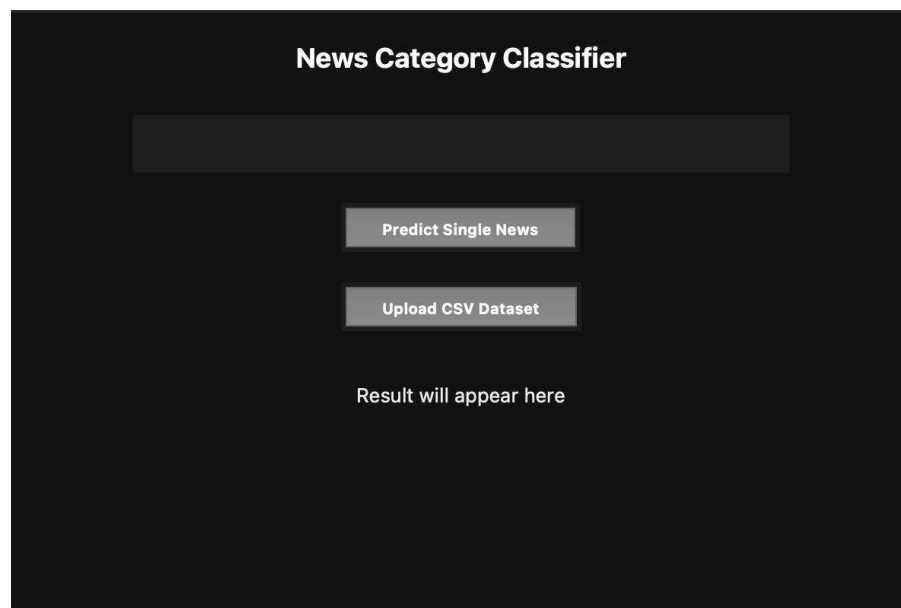
Bulk predictions saved in:

output_predictions.csv

KEY IMPLEMENTATIONS OUTLINE OF THE SYSTEM

- TF-IDF feature extraction
 - Logistic Regression model
 - Dark-themed Tkinter GUI
 - CSV upload functionality
 - Output file generation
-

SCREENSHOTS



SIGNIFICANT PROJECT OUTCOMES

- Accurate classification of news headlines
 - Efficient handling of bulk data
 - User-friendly interface
 - Practical implementation of NLP concepts
-

TESTING AND REFINEMENT

- Tested with 100+ news headlines
 - Verified predictions across categories
 - Improved preprocessing for better accuracy
-

PROJECT APPLICABILITY ON REAL-WORLD APPLICATIONS

- News websites
 - Content recommendation systems
 - Social media platforms
 - Information filtering systems
-

CONTRIBUTION / FINDINGS OF THE PROJECT

- Demonstrated how similarity-based ML works
- Showed importance of feature engineering
- Highlighted real-world use of APIs
- Built a complete AI-based application

LIMITATION / CONSTRAINTS OF THE SYSTEM

- No user personalization
- Depends on dataset quality
- Limited recommendation diversity
- Requires internet for API

CONCLUSION

The project successfully demonstrates a movie recommendation system using machine learning techniques. It provides relevant suggestions based on content similarity and enhances user experience through a graphical interface and real-time data.

FUTURE ENHANCEMENTS

- Add collaborative filtering
- Improve UI design
- Deploy as web app (Streamlit/Flask)
- Add user login system
- Use deep learning models

REFERENCES

- Scikit-learn Documentation
- Pandas Documentation
- OMDb API Documentation
- Python Tkinter Documentation